

DSA1101

Introduction to Data Science

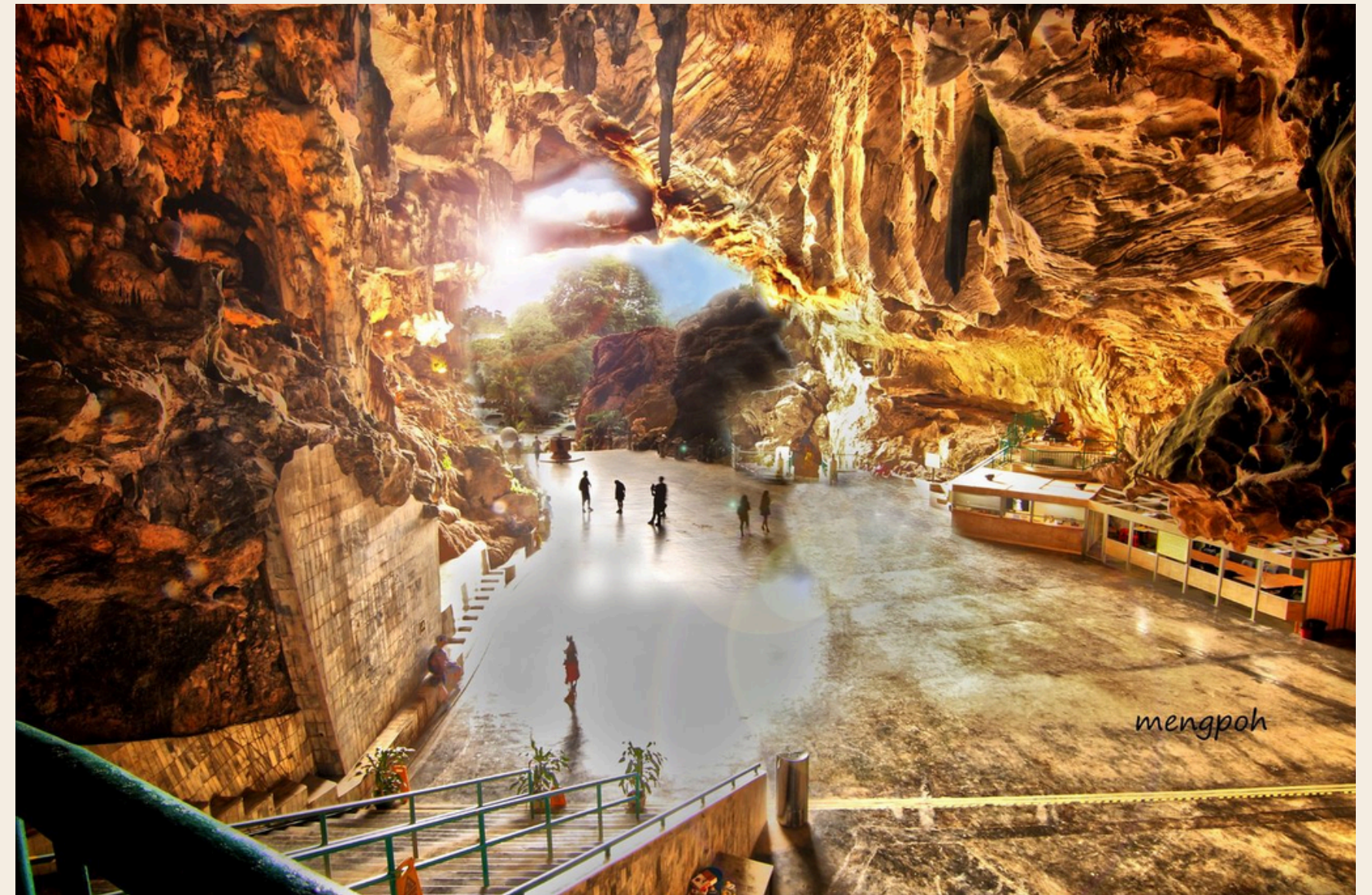
Week 11 Tutorial 9

TODAY'S AGENDA

1. Quick Revision
2. On-site Questions Attempt
3. On-site Questions Discussion
4. Off-site Questions Discussion



WHAT IS EVERYONE DOING THIS BREAK?

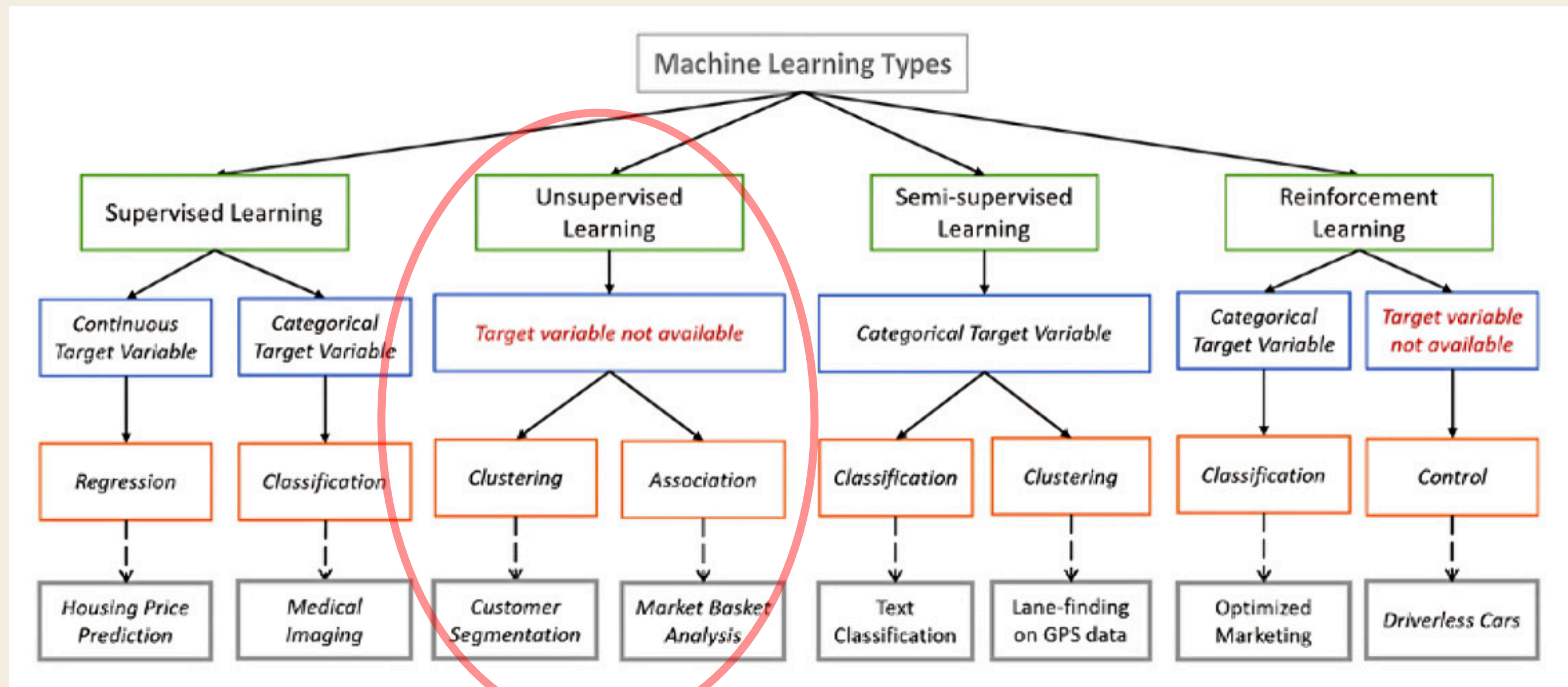


I'm going to Ipoh

QUICK REVISION

UNSUPERVISED LEARNING

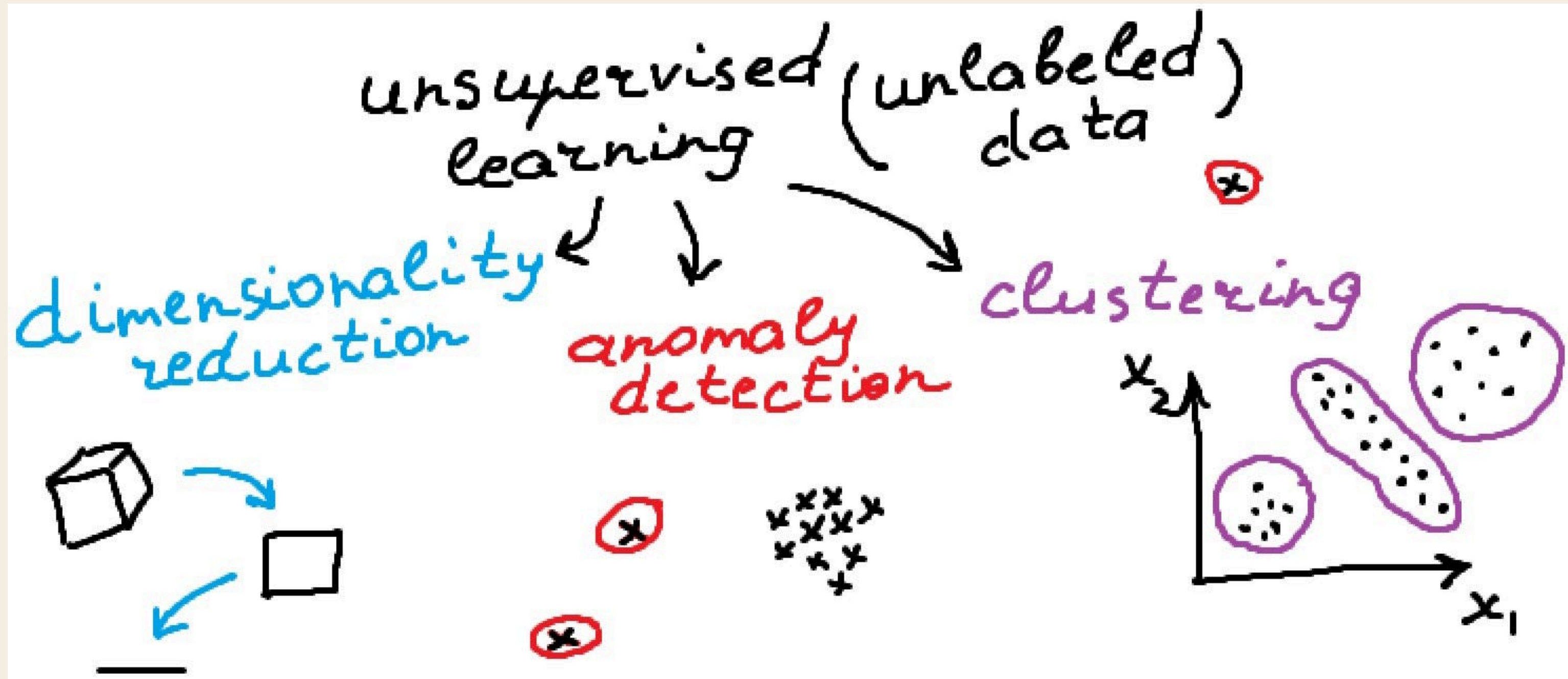
Algorithm that learns without labelled data (no response variable given!)



Useful when labeled data is unavailable or too costly (which is very common irl!)

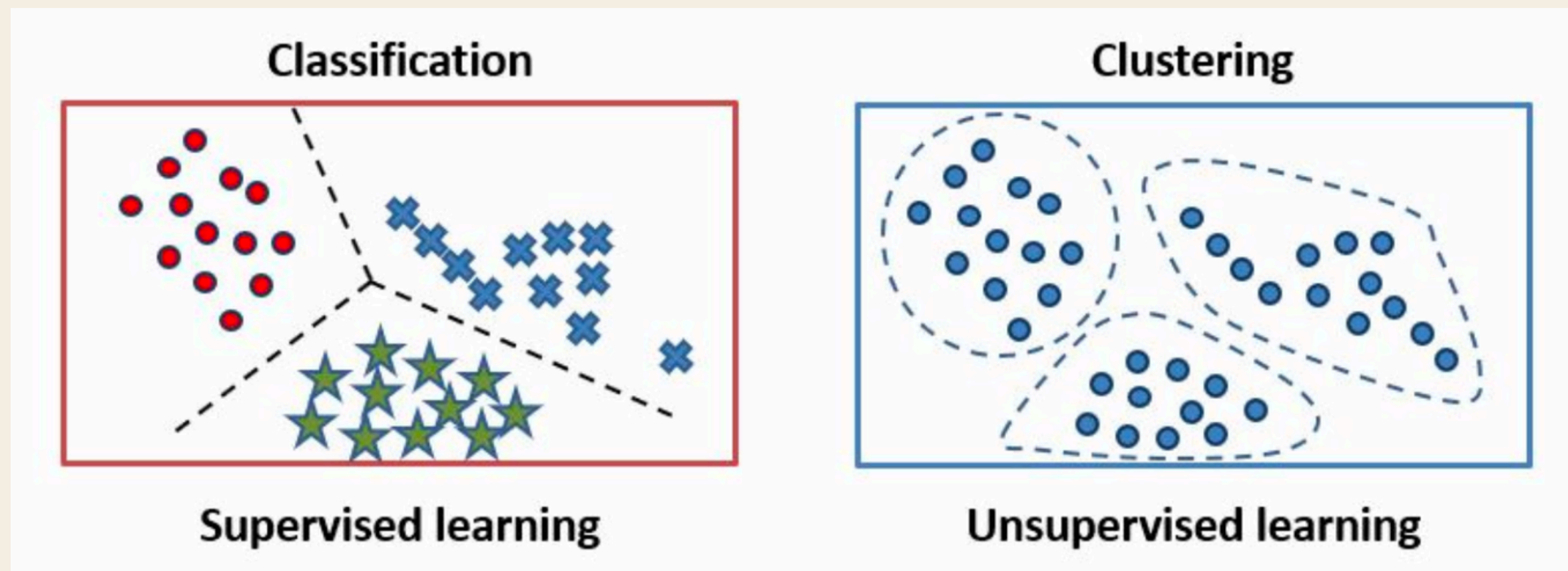
UNSUPERVISED LEARNING

More type of unsupervised learning!



CLUSTERING

- Clustering finds natural groupings in data **based on similarity**
 - Even when we don't know the labels
- Used in: Customer segmentation, image compression etc

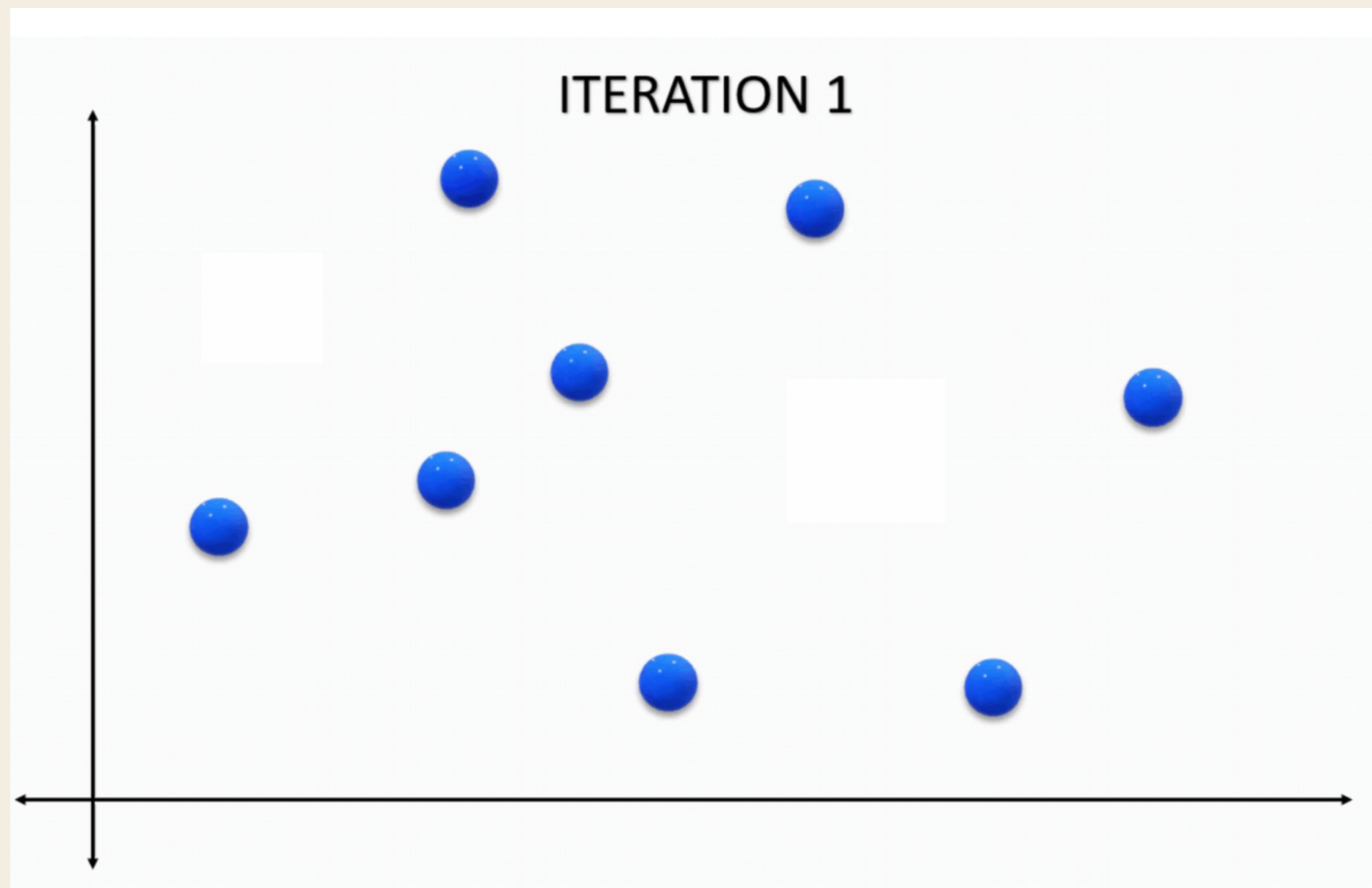


Clustering doesn't tell you what each group is - it just groups similar points together. You'll need to decide if those groupings are useful or interpretable.

K-MEANS CLUSTERING

Goal: Partition data into K clusters by minimizing the distance between points and their assigned centroid.

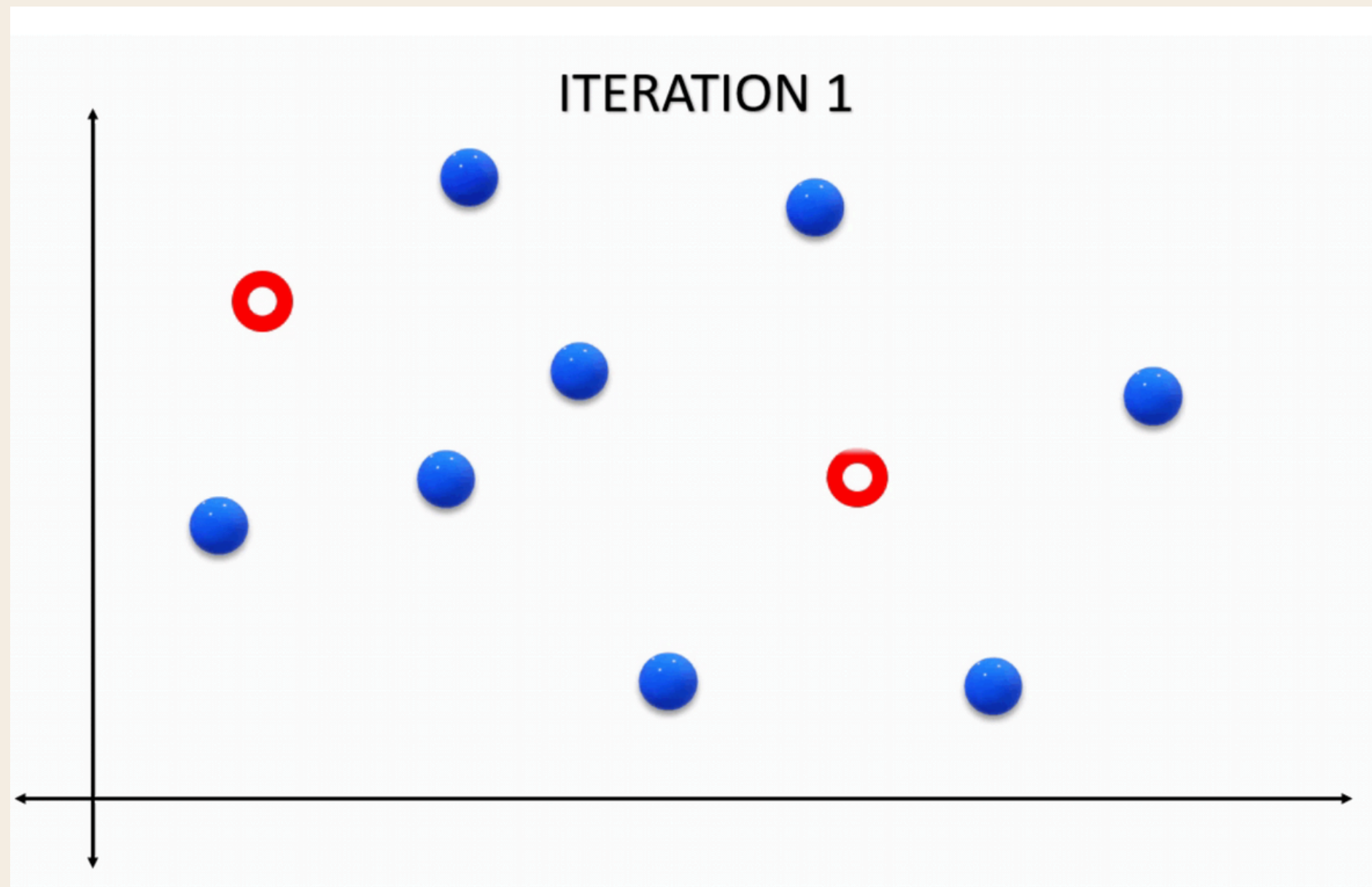
Step 0: Say you have these data points



K-MEANS CLUSTERING

Goal: Partition data into K clusters by minimizing the distance between points and their assigned centroid.

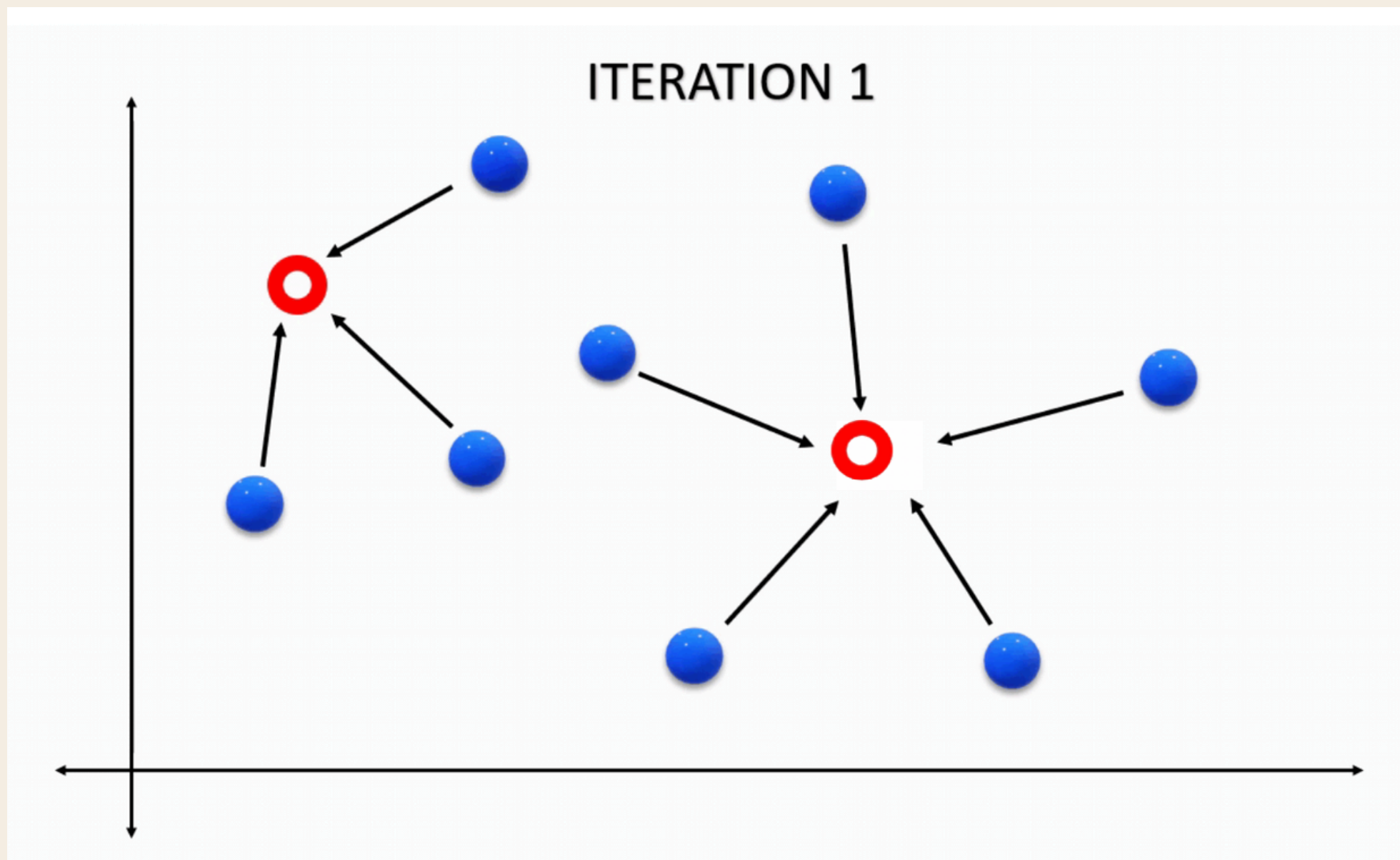
Step 1: Initialize centroids (most of the time it's random initialization)



K-MEANS CLUSTERING

Goal: Partition data into K clusters by minimizing the distance between points and their assigned centroid.

Step 2: Calculate **Euclidean Distance** between points and centroids

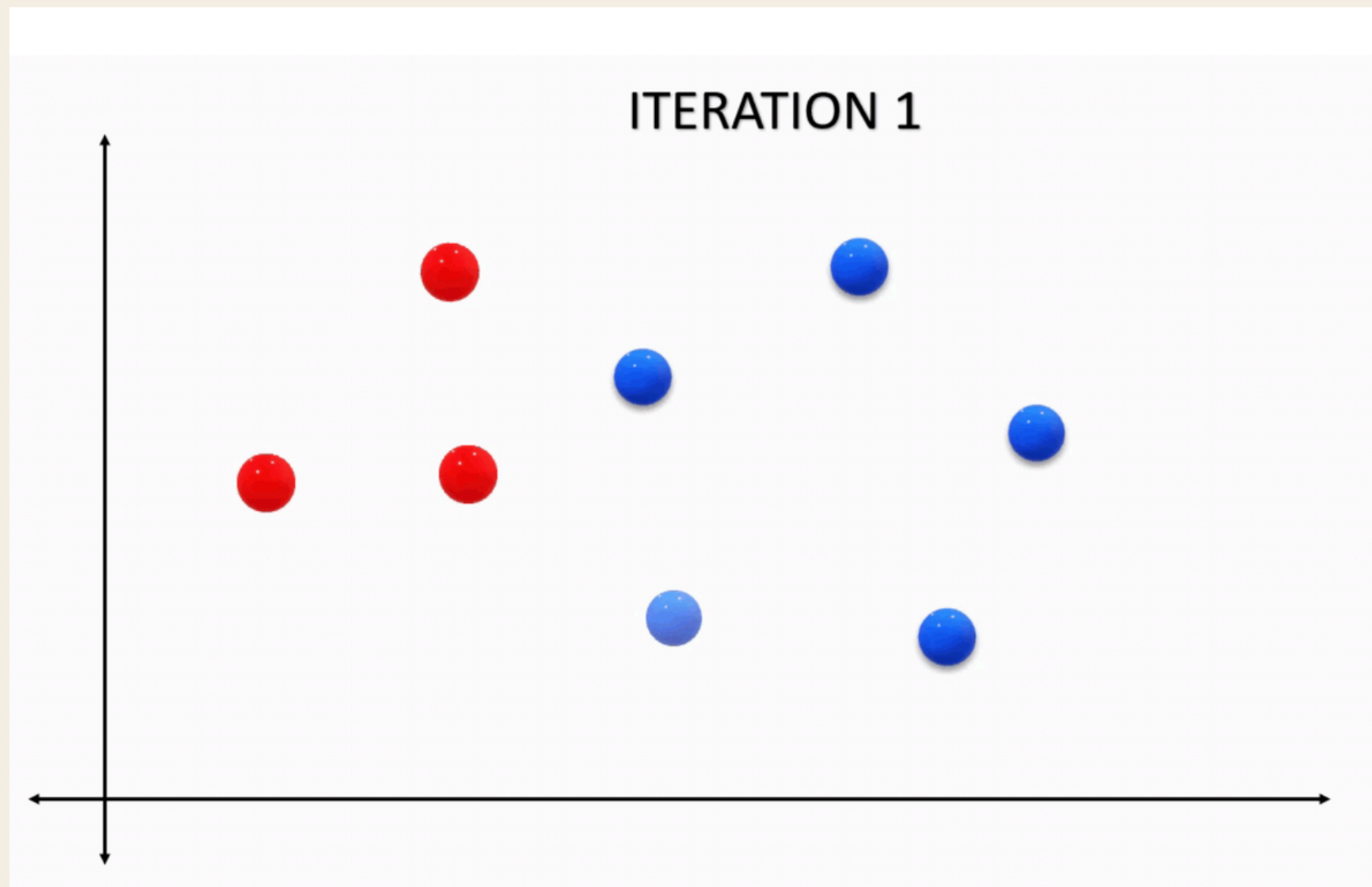


Note: This diagram is slightly inaccurate. We should calculate euclidean distance for all points to each centroids

K-MEANS CLUSTERING

Goal: Partition data into K clusters by minimizing the distance between points and their assigned centroid.

Step 3: Assign each point to the cluster that it's closest centroid represents



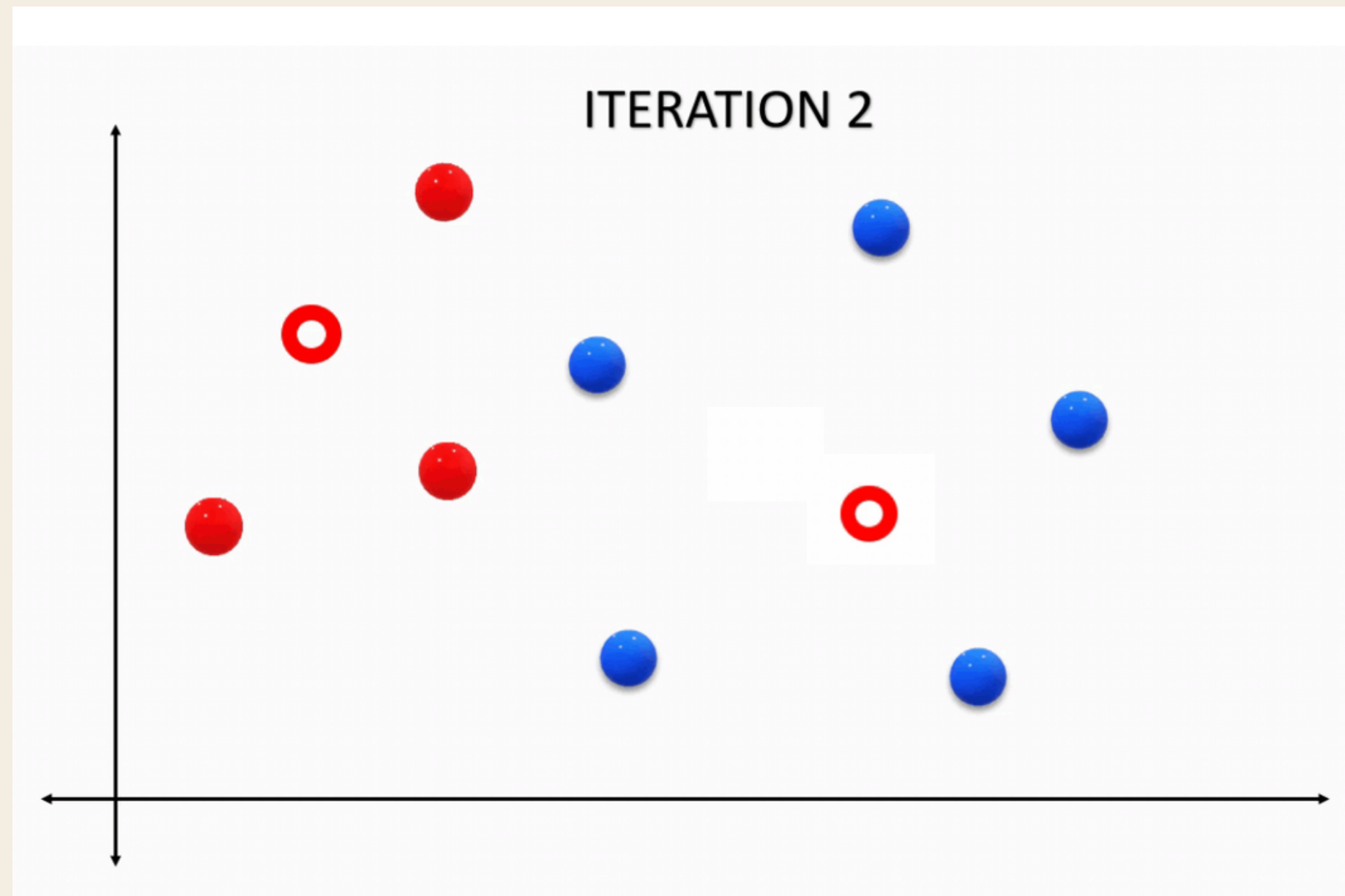
Note: This diagram is slightly inaccurate. We should calculate euclidean distance for all points to each centroids

Then based on the euclidean distances, each point will be assign to a cluster that its closest centroid represents

K-MEANS CLUSTERING

Goal: Partition data into K clusters by minimizing the distance between points and their assigned centroid.

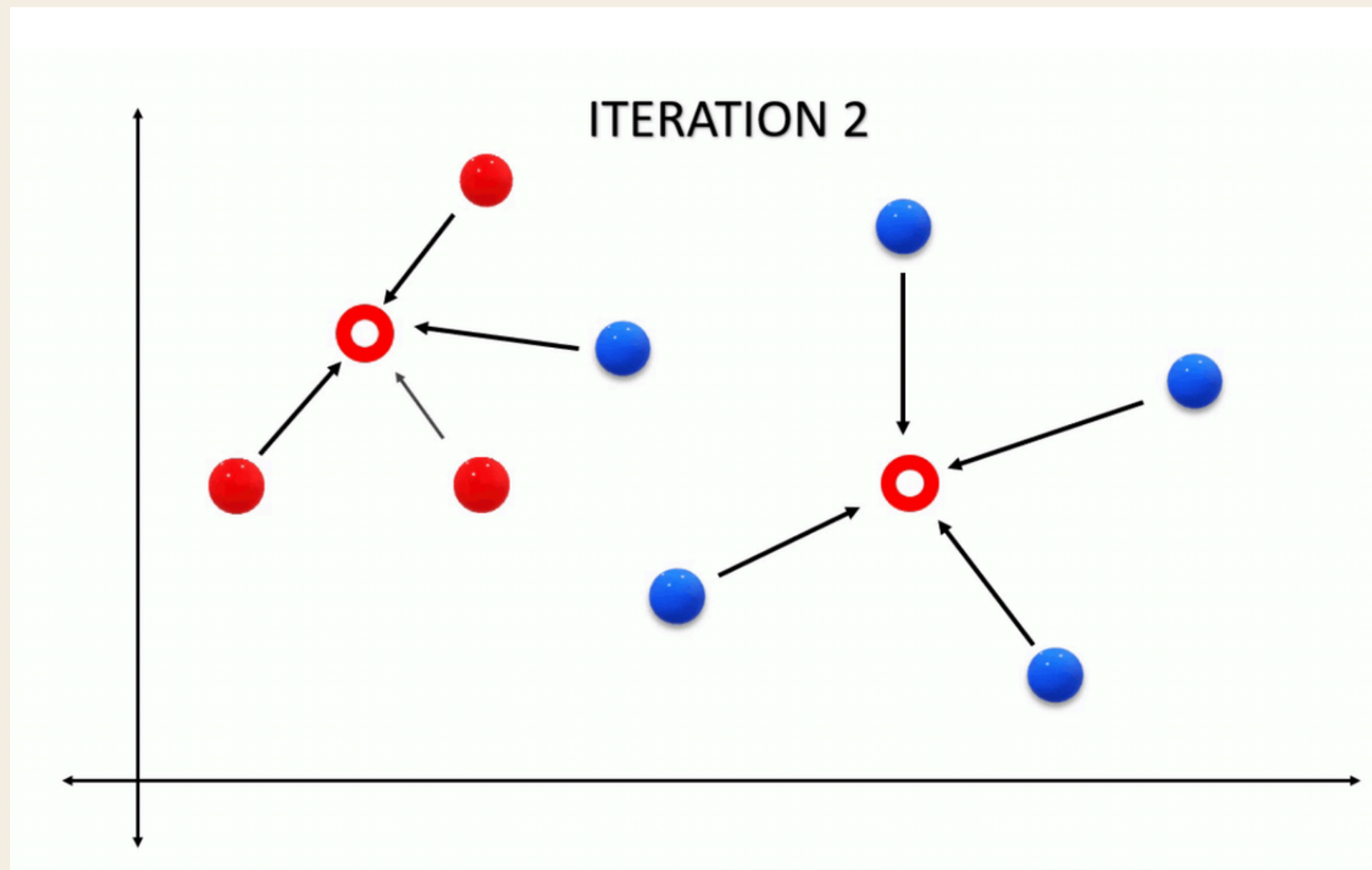
Step 4: Move each centroid to the mean of its assigned points



K-MEANS CLUSTERING

Goal: Partition data into K clusters by minimizing the distance between points and their assigned centroid.

Step 5: Calculate Euclidean Distance between points and newly updated centroids

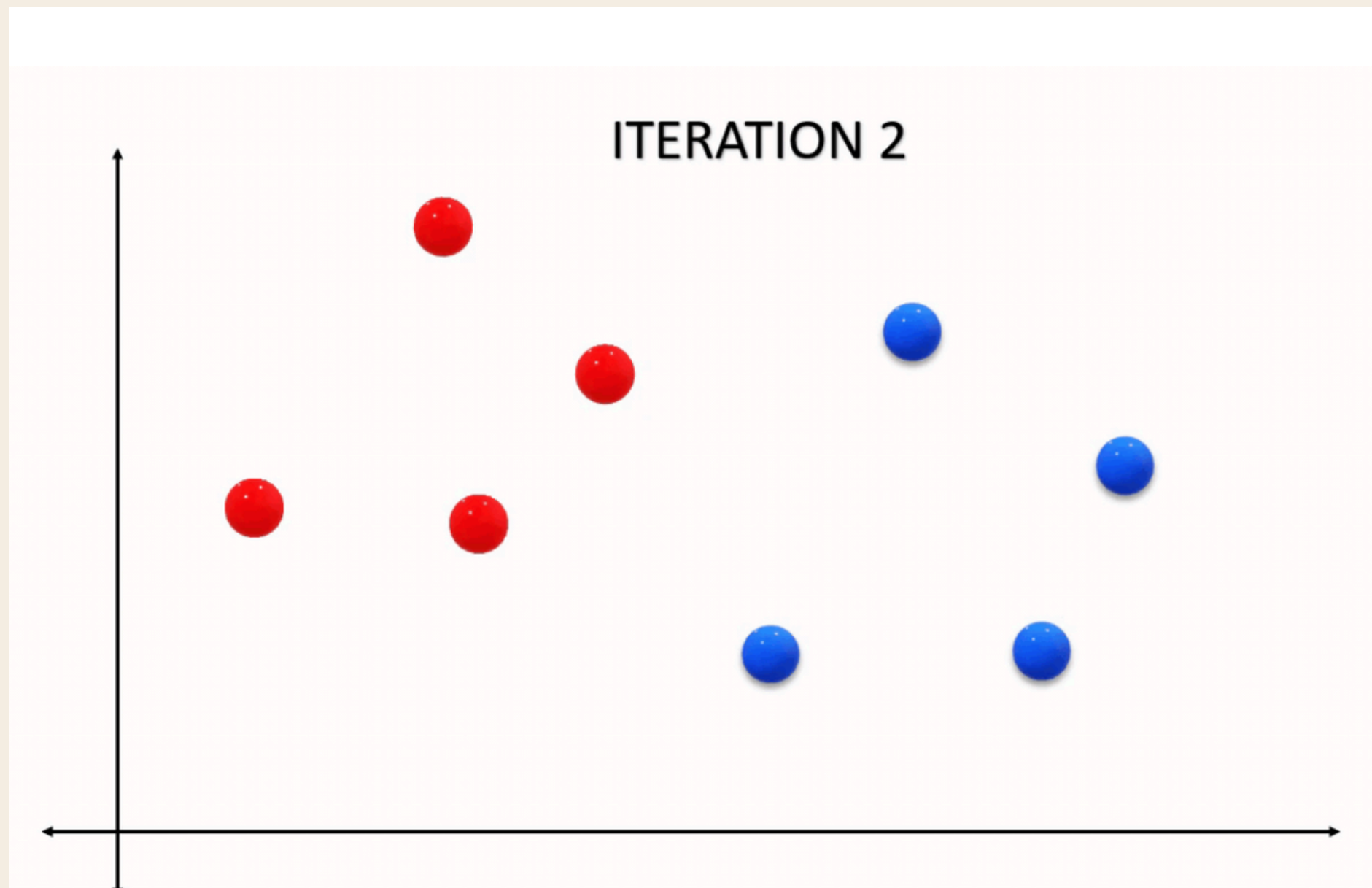


Note: This diagram is slightly inaccurate. We should calculate euclidean distance for all points to each centroids

K-MEANS CLUSTERING

Goal: Partition data into K clusters by minimizing the distance between points and their assigned centroid.

Step 6: Reassign each point to the cluster that it's closest centroid represents

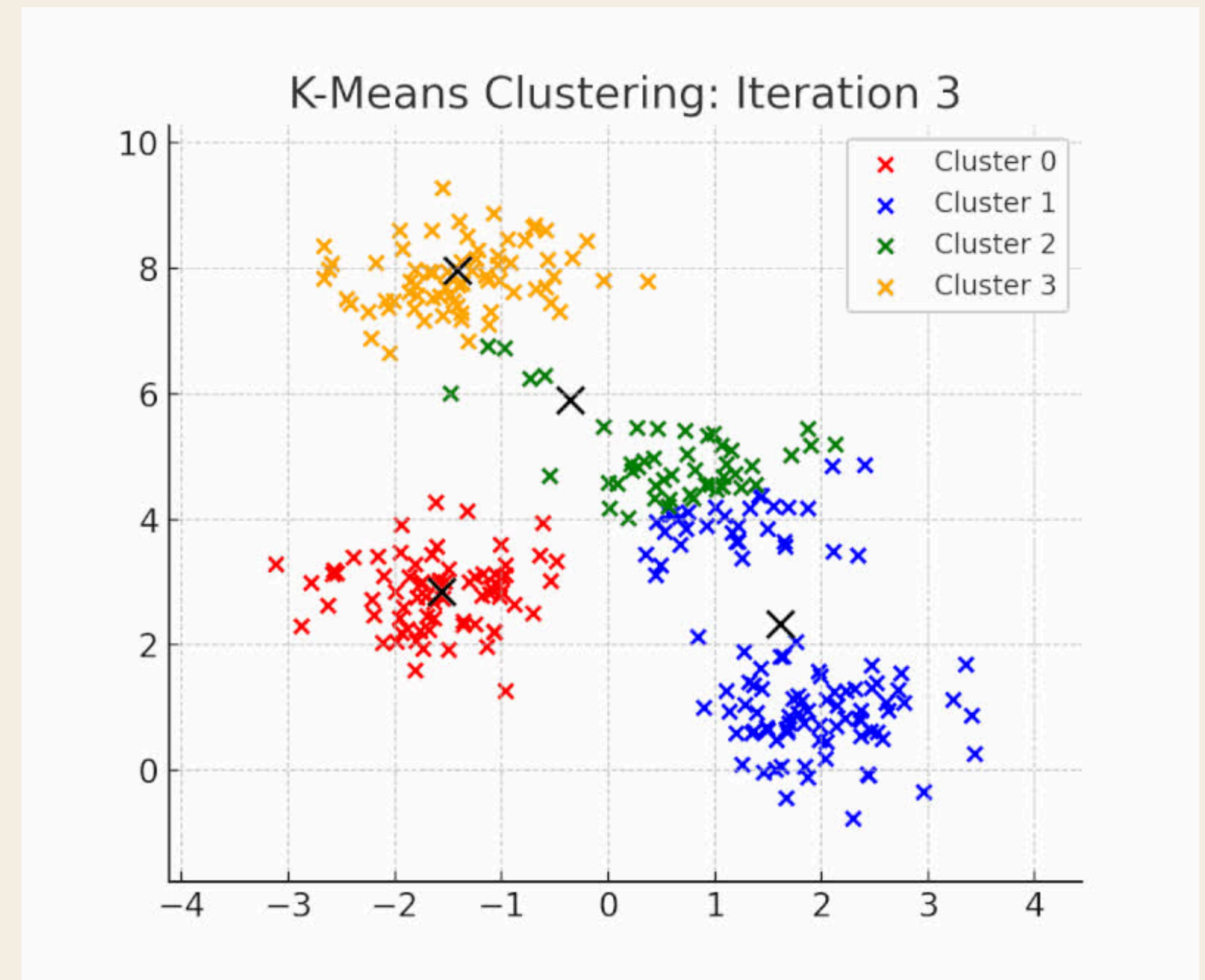


K-MEANS CLUSTERING

Goal: Partition data into K clusters by minimizing the distance between points and their assigned centroid.

Repeat this process until convergence

- centroids no longer change (cluster assignments stop changing)
- maximum iterations is reached



K-MEANS CLUSTERING

Qn: Do we need to do feature scaling for K-Means?

Yes. K-means relies on **distance calculations**, so features with larger ranges can dominate the distance measure if the data is not scaled

Qn: Can we initialise centroids from one of the samples?

Yes. In fact, usual convention (functions in R/Python scikit-learn) is to initialise some points as the centroids randomly.

Qn: If I initialise centroids from one of the samples, then the selected points will move around during K-means?

No. The data points remain fixed. The centroid starts as a **copy of a selected data point**, and this centroid moves to the mean of points that share its cluster

WITHIN-CLUSTER SUM OF SQUARES

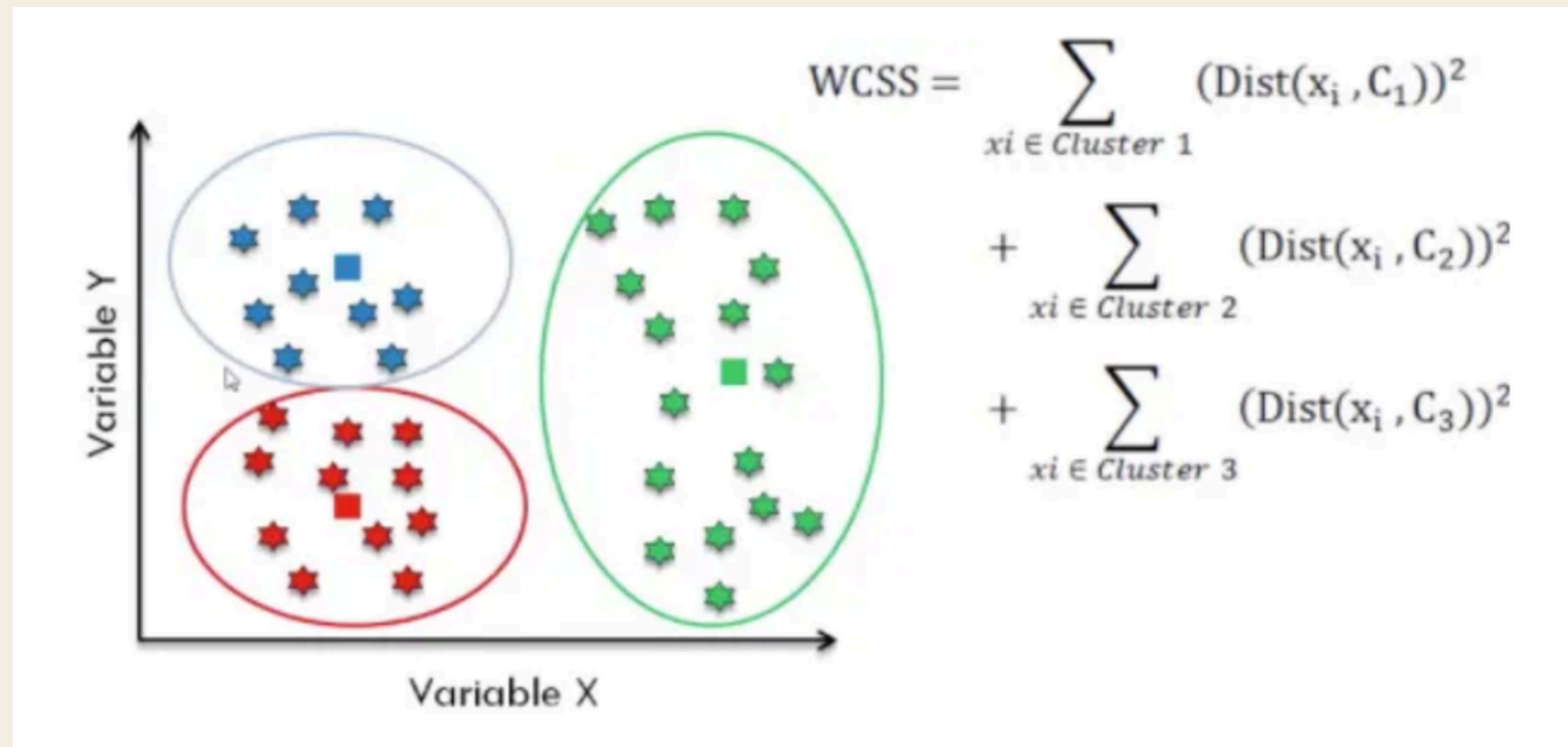
Within-Cluster Sum of Squares (WCSS) is the sum of squared Euclidean distances between each data point and the centroid of its assigned cluster

$$WCSS = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

C_i = Cluster i

μ_i = Centroid of Cluster i

x = Data point of Cluster i



We often evaluate the quality of a clustering using

WCSS (**smaller is better**)

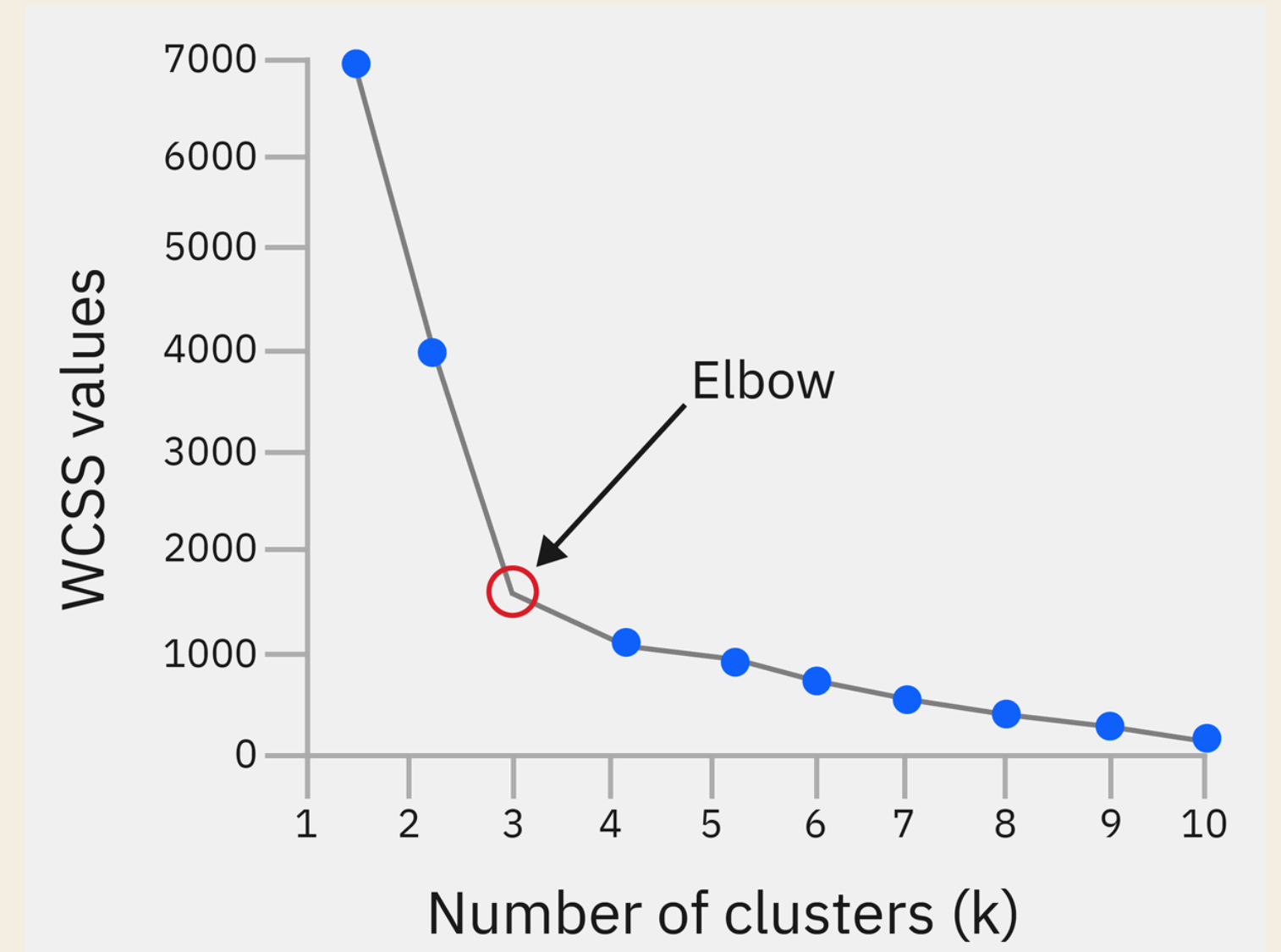
HOW TO CHOOSE K?

1. Elbow Method

- Compute WCSS for different values of K
- Choose K where the decrease in WCSS starts to slow down

Increasing K always reduces WCSS, but too large K may produce meaningless clusters

2. Choose K manually (based on domain knowledge)



K-MEANS CLUSTERING IN R

We use the `kmeans()` function in R!

```
kmeans(x = data, centers = k, iter.max = 10, nstart = 1)
```

Before that

Similar to KNN, we need to do some preparation before calling `kmeans()`

- (1) We need to set seed (randomization present)
- (2) all columns in “data” must be numerical (distance calc)
- (3) data must be standardized

Very important!!!

K-MEANS CLUSTERING IN R

```
kmeans(x = data, centers = k, iter.max = 10, nstart = 1)
```

x	required	A matrix or data frame containing your data (feature variables) • Must only contain numerical variables!
centers	required	Number of clusters (k), OR a set of initial centroid coordinates
iter.max	optional (default iter.max = 10)	Maximum number of iterations before the algorithm stops • If model has not converged but iteration hits iter.max, it'll stop
nstart	optional (default nstart = 1)	Number of times you want the K-means to run again with different centroid initialization • Best result is returned (smallest WCSS)

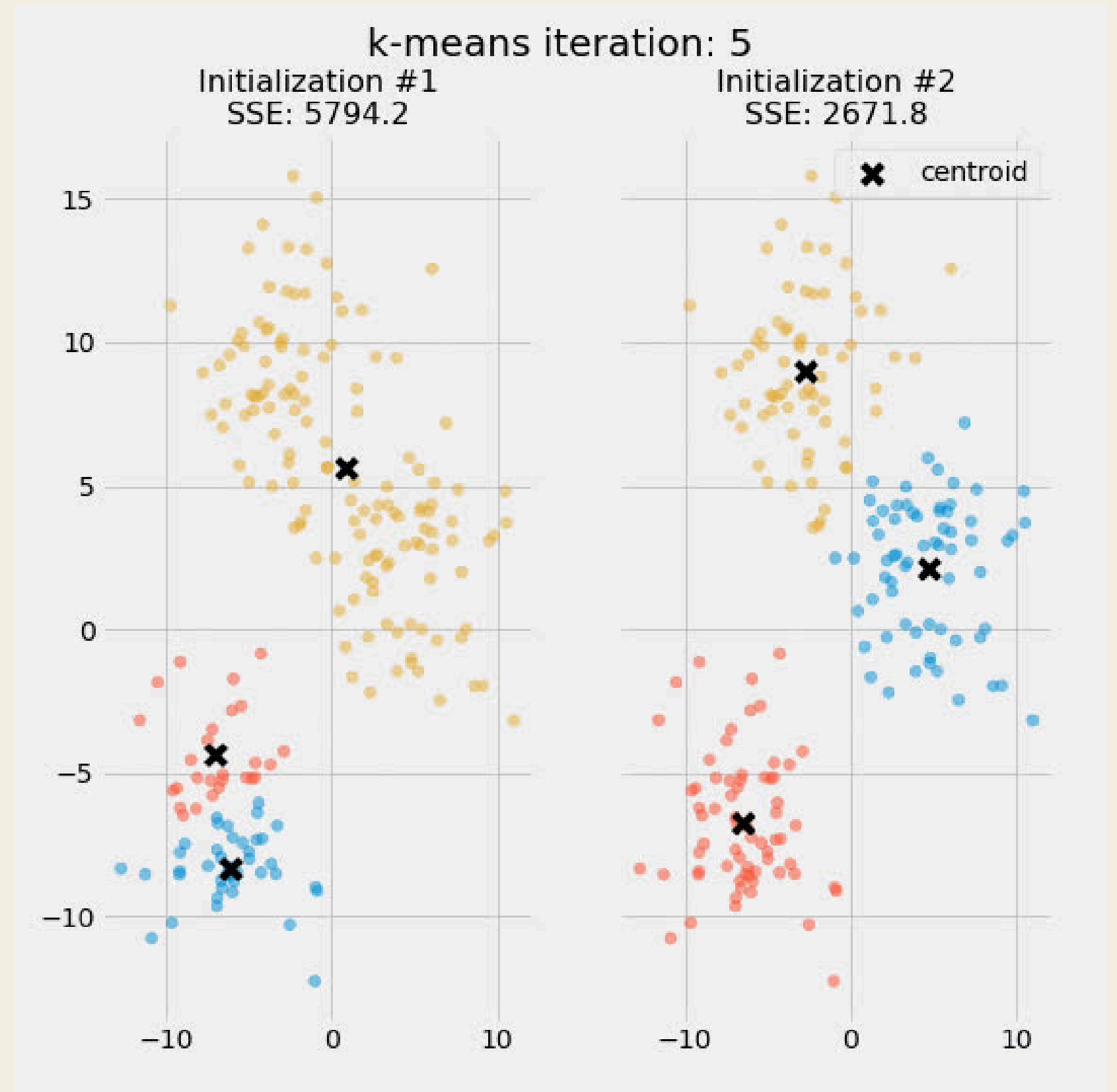
WHY DO WE NEED `nstart`?

Important:

K-means is highly sensitive to initial placement of centroids!!

`nstart` will run K-means multiple times with different centroid initializations

And picks the one that gives the best results (smallest WCSS)

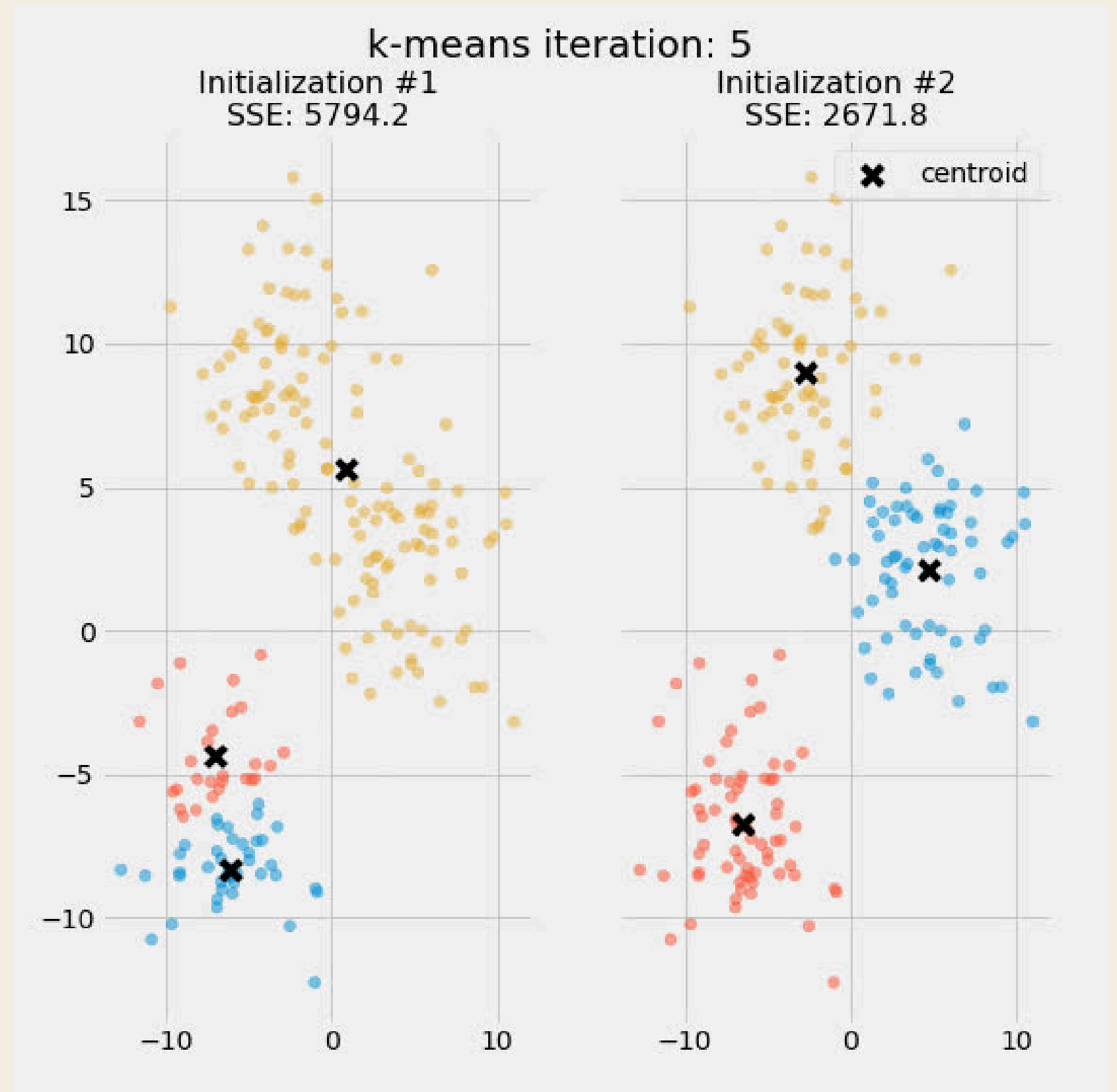


WHY DO WE NEED `nstart`?

Recall that in R, **centroids are initialized to some random point from your data.**

So ideally, *nstart* should be lesser than the number of data points in your data (but more than 1)

A good range is 20 to 50 (depending on size of your data)



COMPONENTS OF `kmeans()`

Viewing what components are available:

```
names(your_kmeans_model)
```

```
[1] "cluster"      "centers"      "totss"        "withinss"
[5] "tot.withinss" "betweenss"    "size"         "iter"
[9] "ifault"
```

COMPONENTS OF `kmeans()`

You can also call `?kmeans` and scroll down to find the description

<code>cluster</code>	A vector of integers (from 1:k) indicating the cluster to which each point is allocated.
<code>centers</code>	A matrix of cluster centres.
<code>totss</code>	The total sum of squares.
<code>withinss</code>	Vector of within-cluster sum of squares, one component per cluster.
<code>tot.withinss</code>	Total within-cluster sum of squares, i.e. <code>sum(withinss)</code> .
<code>betweenss</code>	The between-cluster sum of squares, i.e. <code>totss-tot.withinss</code> .
<code>size</code>	The number of points in each cluster.
<code>iter</code>	The number of (outer) iterations.
<code>ifault</code>	integer: indicator of a possible algorithm problem – for experts.

We mostly only care about these 3!

`model$centers`: Final centroid coordinates

`model$tot.withinss`: Total within-cluster sum of squares

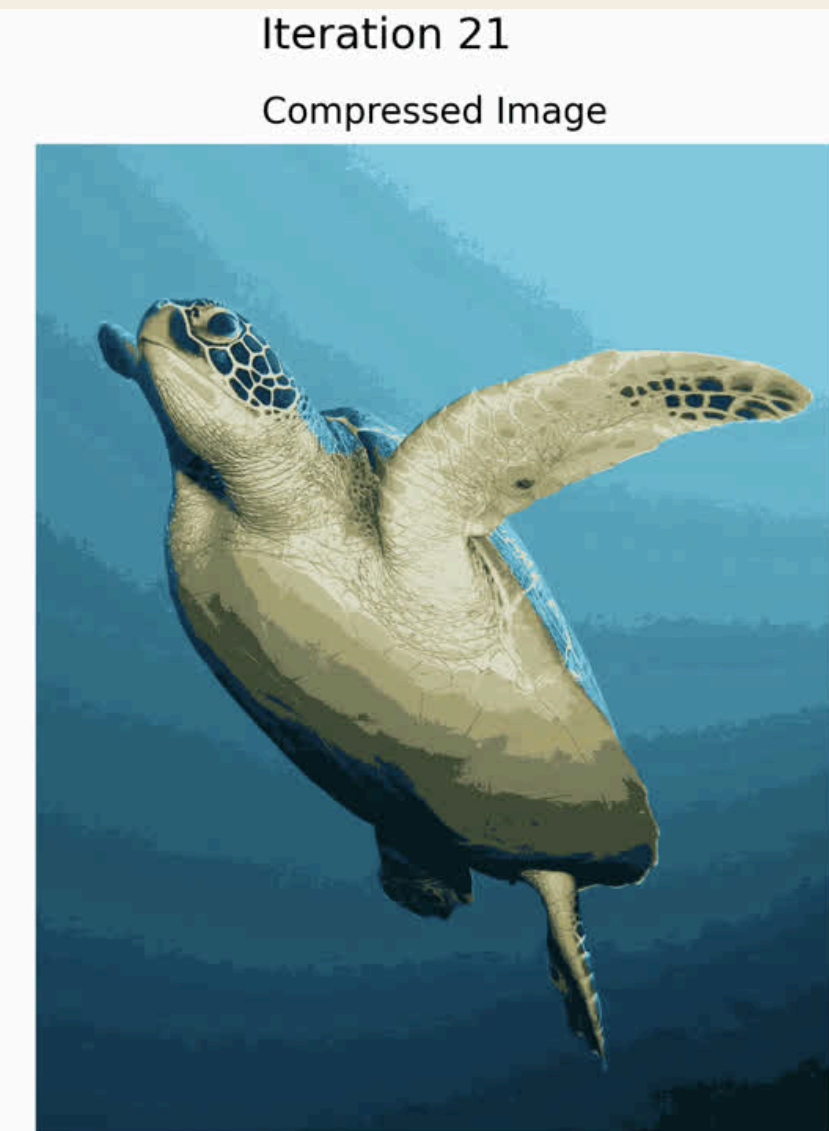
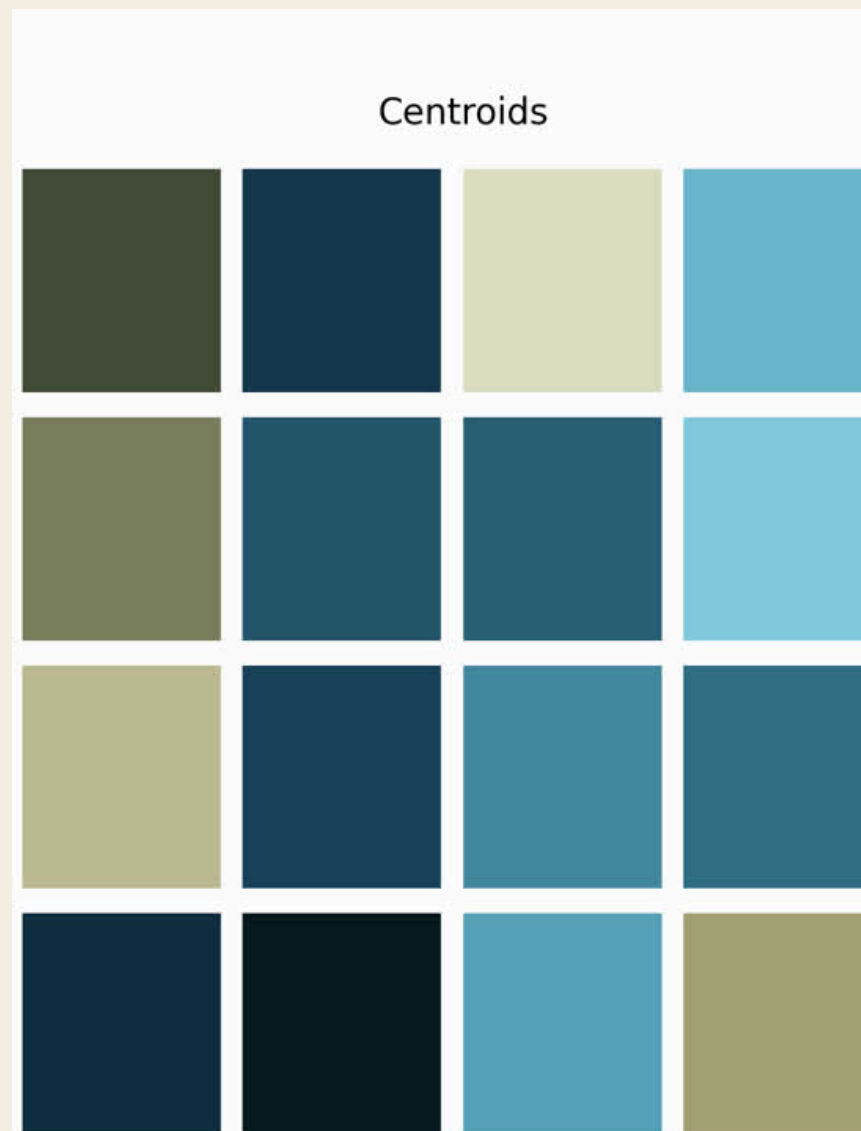
`model$size`: Number of points in each

cluster

```
model$size  
# e.g. 47 50 53
```

CLUSTERING IN IMAGE COMPRESSION

- K-Means clustering groups similar colors (RGB values) together into k clusters.
- Instead of storing the full RGB value for each pixel, we:
 - Replace each pixel's color with the nearest cluster center (centroid)
 - Only store the k color centroids + the pixel's cluster label.



ON-SITE QUESTIONS

ON-SITE QUESTIONS

(K-Means)

Consider the famous Iris Flower Data set which was first introduced in 1936 by the famous statistician Ronald Fisher. This data set consists of observations from flowers of Iris species. For each observation, four features were measured: the flower's length and width of the sepals and petals (in cm).

The data set is given in a file named `iris.csv`.

1. Use K-means clustering method to cluster all the flowers into k groups where $k = 1, 2, 3, \dots, 10$, where for each value of k the value of WSS - the within sum of squares is obtained.

So for this question you should get a vector of 10 WSS values :)

Discuss with your group! Time limit: 3.5 mins

ON-SITE PREP AND INSPECT

```
iris = read.csv("Data/iris.csv")
```

```
head(iris)
```

```
str(iris)
```

```
dim(iris) # 150 rows
```

```
> head(iris)
```

	sepal.length	sepal.width	petal.length	petal.width	class
1	5.1	3.5	1.4	0.2	Iris-setosa
2	4.9	3.0	1.4	0.2	Iris-setosa
3	4.7	3.2	1.3	0.2	Iris-setosa
4	4.6	3.1	1.5	0.2	Iris-setosa
5	5.0	3.6	1.4	0.2	Iris-setosa
6	5.4	3.9	1.7	0.4	Iris-setosa

ON-SITE PREP AND INSPECT

```
> str(iris)
'data.frame':   150 obs. of  5 variables:
 $ sepal.length: num  5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
 $ sepal.width : num  3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
 $ petal.length: num  1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
 $ petal.width : num  0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
 $ class       : chr  "Iris-setosa" "Iris-setosa" "Iris-setosa" "Iris-setosa"
```

All feature variables are numerical

No changes to be made

Recall that K-means, like KNN, only takes numerical inputs

ON-SITE QUESTION 1

Q: Run K-means for iris dataset from $k = 1 \dots 10$ and obtain the WSS values

Step 0: Set seed (impt even though official ans does not have)

```
set.seed(1505)
```

Step 1: Standardize the input variables

```
scaled_data = scale(iris[1:4])  
head(scaled_data)
```

```
> head(scaled_data)  
      sepal.length sepal.width petal.length petal.width  
[1,]   -0.8976739    1.0286113   -1.336794   -1.308593  
[2,]   -1.1392005   -0.1245404   -1.336794   -1.308593  
[3,]   -1.3807271    0.3367203   -1.393470   -1.308593  
[4,]   -1.5014904    0.1060900   -1.280118   -1.308593
```

ON-SITE QUESTION 1

Q: Run K-means for iris dataset from $k = 1 \dots 10$ and obtain the WSS values

Step 2: Perform K-means from $k = 1 \dots 10$

```
K = 10
```

```
wss = numeric(K) # to store wss values
```

```
for (k in 1:K) {
```

(1) Run k-means on that specific k your loop is on

```
  model <- kmeans(scaled_data, centers = k, nstart = 20)
```

```
  wss[k] = model$tot.withinss
```

(2) Obtain the WSS of the model

```
}
```

ON-SITE QUESTION 1

Q: Run K-means for iris dataset from $k = 1 \dots 10$ and obtain the WSS values

Step 2: Perform K-means from $k = 1 \dots 10$

```
> WSS
[1] 596.00000 222.24046 140.02604 113.60040 90.39576 81.14577 70.38680
[8] 62.50157 54.70555 47.04607
```

Note: You might get different values from me if you set different
seed/n_start

ON-SITE QUESTIONS

2. Write code to obtain the plot of WSS against k . Which value of k would you choose as the number of clusters for all the observations in the data set? Explain.
3. With the value of k chosen above, report the centroids of all the clusters and the number of the observations in each cluster.

Hint for Q2: Use Elbow method

Hint for Q3: Recall calling components of model

Discuss with your group! Time limit: 3.5 mins

ON-SITE QUESTION 2

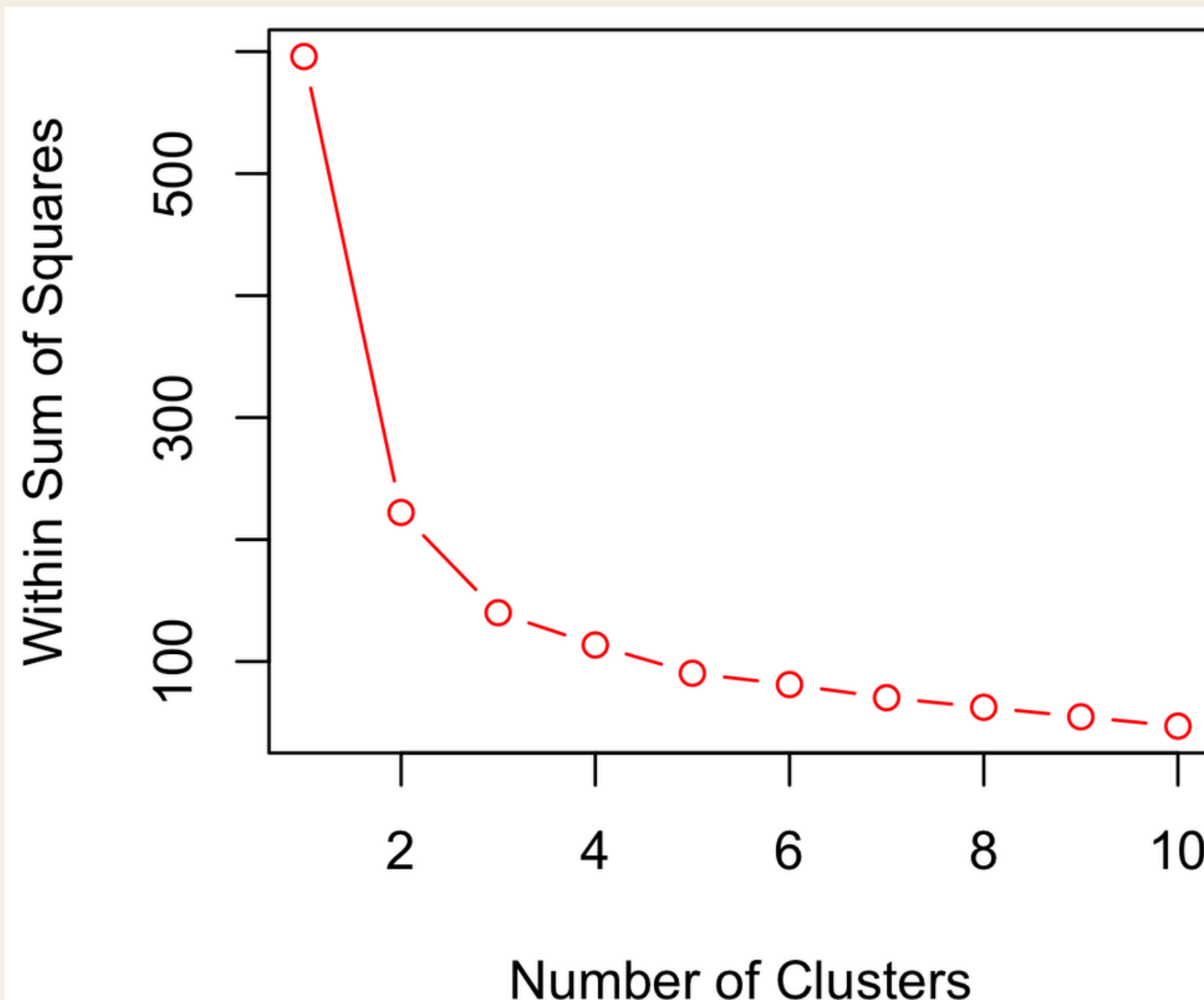
Q: Plot the WSS obtained against k. State which k you would choose and briefly explain why

To plot WSS against k:

```
plot(x = 1:K, y = wss,  
     col = "red", type = "b",  
     xlab = "Number of Clusters",  
     ylab = "Within Sum of Squares")
```

ON-SITE QUESTION 2

Q: Plot the WSS obtained against k. State which k you would choose and briefly explain why



Which k would you choose?

k = 3 seems like a good choice as reduction in WSS from k = 3 to k = 4 is not too significant

ON-SITE QUESTION 3

Q: With the best k chosen, report centroids of each cluster and number of data points in each cluster

Say we chose $k = 3$: First fit the model again

```
# Apply K-means with  $k = 3$  (best  $k$ )  
kout = kmeans(scaled_data,  
               centers = 3,  
               nstart = 20)
```

ON-SITE QUESTION 3

Q: With the best k chosen, report centroids of each cluster and number of data points in each cluster

Say we chose $k = 3$: Next, obtain the centroids using `$centers`

```
# Obtain the 3 centroids  
kout$centers
```

```
> kout$centers  
  sepal.length sepal.width petal.length petal.width  
1    1.13217737    0.0962759    0.9929445    1.0137756  
2   -1.01119138    0.8394944   -1.3005215   -1.2509379  
3   -0.05005221   -0.8773526    0.3463713    0.2811215
```

Notice that the centroids coordinates are the standardized version!!

ON-SITE QUESTION 3

Q: With the best k chosen, report centroids of each cluster and number of data points in each cluster

EXTRA!!

You can unstandardize it for interpretation

$$z = \frac{x - \mu}{\sigma}$$

μ = Mean

σ = Standard Deviation

```
centers_unscaled = sweep(kout$centers,  
                          MARGIN = 2,  
                          attr(scaled_data, "scaled:scale"), "*")  
centers_unscaled = sweep(centers_unscaled,  
                          MARGIN = 2,  
                          attr(scaled_data, "scaled:center"), "+")  
centers_unscaled
```

Don't really need to know this for this course ;)

ON-SITE QUESTION 3

Q: With the best k chosen, report centroids of each cluster and number of data points in each cluster

EXTRA!!

After unstanrdizing:

```
> centers_unscaled
  sepal.length sepal.width petal.length petal.width
1    6.780851    3.095745    5.510638    1.972340
2    5.006000    3.418000    1.464000    0.244000
3    5.801887    2.673585    4.369811    1.413208
```

Don't really need to know this for this course ;)

ON-SITE QUESTION 3

Q: With the best k chosen, report centroids of each cluster and number of data points in each cluster

No extra

Number of points in each cluster using $\$size$:

```
> kout$size  
[1] 47 50 53
```

OFF-SITE QUESTION 1

Today we demonstrate
this using slides

OFF-SITE QUESTION 1A

1. Suppose we have data for five objects on two features:

object	x_1	x_2
A	1	1
B	1.5	2
C	3	4
D	3.5	5
E	4.5	5

We set $k = 2$ to cluster the five data points into two clusters, $\mathcal{P}$ and $\mathcal{Q}$, and initialize the algorithm with the centroids $(x_{1,\mathcal{P}}, x_{2,\mathcal{P}}) = (2, 2)$ and $(x_{1,\mathcal{Q}}, x_{2,\mathcal{Q}}) = (4, 4)$.

- (a) Fill up the following table to identify the objects in each cluster during the first iteration of the k -means algorithm:

cluster	object(s)
$\mathcal{P}$	
$\mathcal{Q}$	

OFF-SITE QUESTION 1A

Initialized centroids: centroid_P = (2,2) centroid_Q = (4,4)

Iteration 1: Get the Euclidean distance of each point to centroid P and centroid Q

In case you forgot:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

object	x_1	x_2	euclidean dist to P	euclidean dist to Q
A	1	1	1.414	4.243
B	1.5	2	0.500	3.202
C	3	4	2.236	1.000
D	3.5	5	3.354	1.118
E	4.5	5	3.905	1.118

OFF-SITE QUESTION 1A

Initialized centroids: **centroid_P = (2,2) centroid_Q = (4,4)**

Iteration 1: Allocate each point to P and Q depending on which centroid they are closer to:

object	x_1	x_2	euclidean dist to P	euclidean dist to Q
A	1	1	1.414	4.243
B	1.5	2	0.500	3.202
C	3	4	2.236	1.000
D	3.5	5	3.354	1.118
E	4.5	5	3.905	1.118

Hence:

cluster	object(s)
P	A, B
Q	C, D, E

OFF-SITE QUESTION 1B

Q: Compute the new centroids for the two clusters

Cluster Assignment:

cluster	object(s)
$\mathcal{P}$	A, B
$\mathcal{Q}$	C, D, E

object	x_1	x_2
A	1	1
B	1.5	2
C	3	4
D	3.5	5
E	4.5	5

$$p: (\text{new-}\mu_1, \text{new-}\mu_2)$$

$$= \left(\frac{1+1.5}{2}, \frac{1+2}{2} \right) = (1.25, 1.5)$$

$$Q: (\text{new-}\mu_1, \text{new-}\mu_2)$$

$$= \left(\frac{3+3.5+4.5}{3}, \frac{4+5+5}{3} \right) = (3.67, 4.67)$$

New centroids:

$$\text{centroid}_P = (1.25, 1.5) \quad \text{centroid}_Q = (3.67, 4.67)$$

OFF-SITE QUESTION 1C

Q: Based on the centroids computed in (b), identify the objects in each cluster during the second iteration of the k-means algorithm.

New centroids: centroid_P = (1.25, 1.5) centroid_Q = (3.67, 4.67)

object	μ_1	μ_2	euclidean dist to new-P	euclidean dist to new-Q
A	1	1	0.559	4.534
B	1.5	2	0.559	3.436
C	3	4	3.052	0.943
D	3.5	5	4.161	0.373
E	4.5	5	4.776	0.898

Just repeat the step of calculating the euclidean distance (this time just to the new centroids)

OFF-SITE QUESTION 1C

Q: Based on the centroids computed in (b), identify the objects in each cluster during the second iteration of the k-means algorithm.

New centroids: centroid_P = (1.25, 1.5) centroid_Q = (3.67, 4.67)

object	μ_1	μ_2	euclidean dist to new-P	euclidean dist to new-Q
A	1	1	0.559	4.534
B	1.5	2	0.559	3.436
C	3	4	3.052	0.943
D	3.5	5	4.161	0.373
E	4.5	5	4.776	0.898

Based on the smallest distance to centroid, assign point to cluster:

cluster	object(s)
P	A, B
Q	C, D, E

The same result as iteration 1, hence we say it converged and stop!

OFF-SITE QUESTION 1D

Q: Calculate the Within Sum of Squares (WSS) for (c).

New centroids: centroid_P = (1.25, 1.5) centroid_Q = (3.67, 4.67)

Point A to centroid P

Point B to centroid P

$$\text{cluster } \mathcal{P}: (1 - 1.25)^2 + (1 - 1.5)^2 + (1.5 - 1.25)^2 + (2 - 1.5)^2 = 0.625$$

$$\text{cluster } \mathcal{Q}: (3 - 3.67)^2 + (4 - 4.67)^2 + (3.5 - 3.67)^2 + (5 - 4.67)^2 + (4.5 - 3.67)^2 + (5 - 4.67)^2 \approx 1.833$$

Point C to centroid Q

Point D to centroid Q

Point E to centroid Q

$$\text{WSS} \approx 0.625 + 1.833 = 2.458$$

OFF-SITE QUESTION 1D

Q: Calculate the Within Sum of Squares (WSS) for (c).

Alternatively, we can use the Euclidean Distance we calculated in (c)

object	x_1	x_2	euclidean dist to new-P	euclidean dist to new-Q
A	1	1	0.559	4.534
B	1.5	2	0.559	3.436
C	3	4	3.052	0.943
D	3.5	5	4.161	0.373
E	4.5	5	4.776	0.898

$$WSS = SS_P + SS_Q$$

$$SS_P = (0.559)^2 + (0.559)^2 \approx 0.625$$

$$SS_Q = (0.943)^2 + (0.373)^2 + (0.898)^2 \approx 1.835$$

$$WSS \approx 0.625 + 1.835 = 2.46$$

Just square it to cancel the square root
from Euclidean Distance!

OFF-SITE QUESTION 2

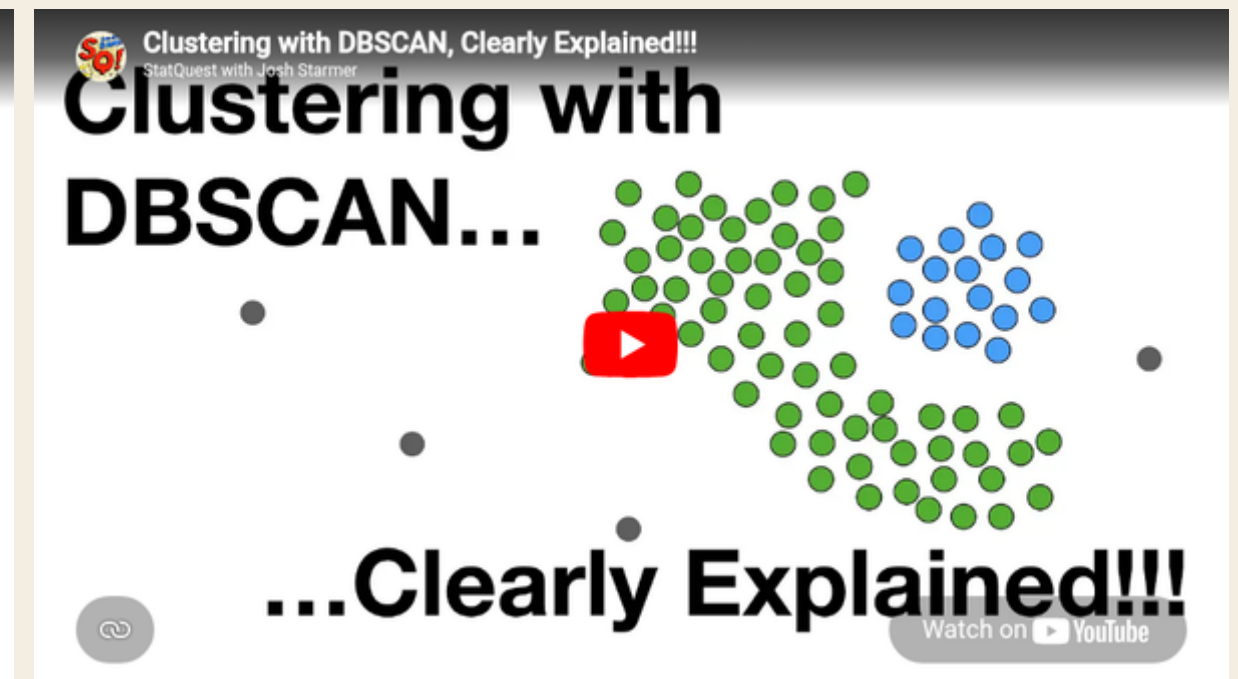
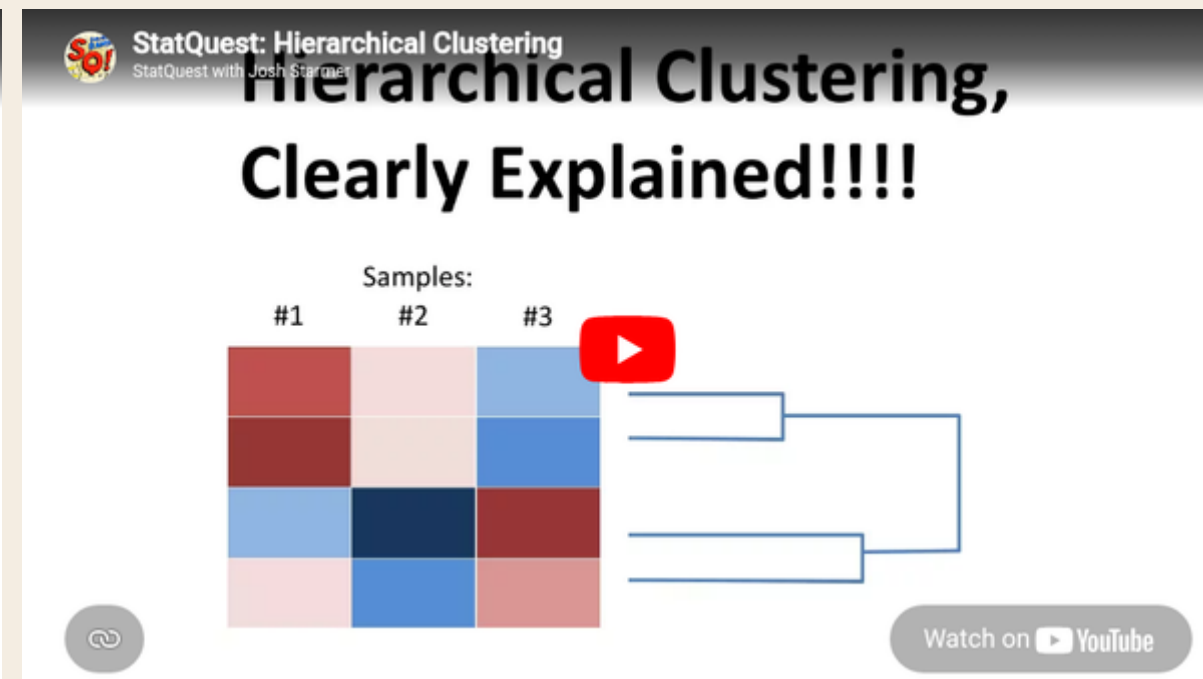
Let's go to code!

Only 1 question to discuss woohooo

Extra!!!

Videos of Clustering – Not just K-means! (by StatQuest):

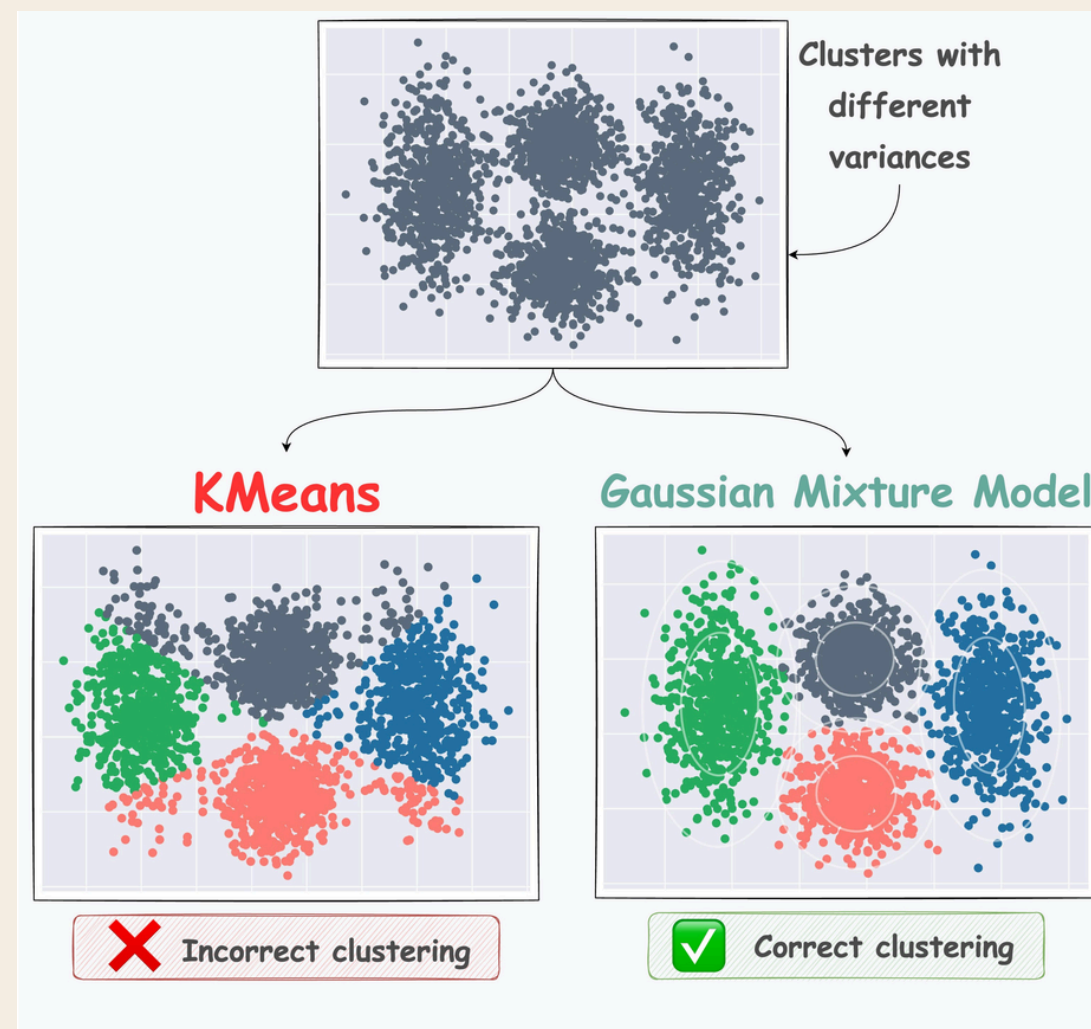
- K-means Clustering: <https://www.youtube.com/watch?v=4b5d3muPQmA>
- Hierarchical Clustering: <https://www.youtube.com/watch?v=7xHsRkOdVwo>
- DBSCAN: <https://www.youtube.com/watch?v=RDZUdRSDOok>



Can watch when you're free during holidays!

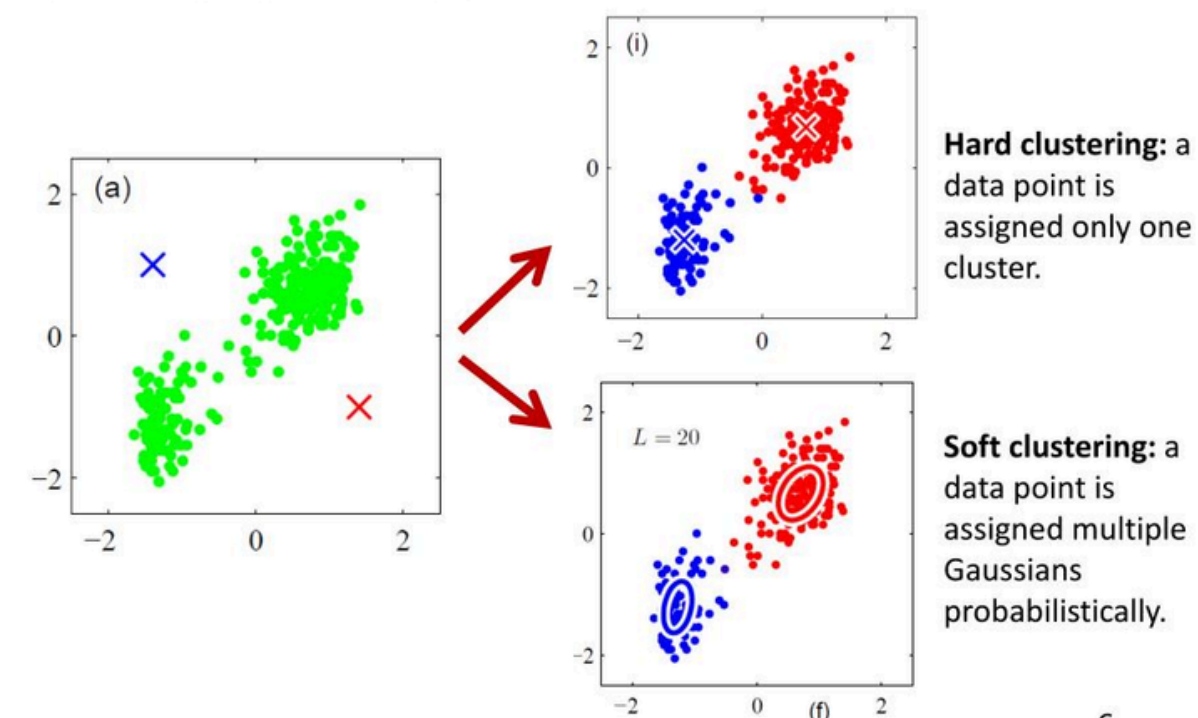
Extra (More advanced)!!!

- Another type of Clustering: Gaussian Mixture Models
- Intro to Dimensionality Reduction
- Deeper into Dimensionality Reduction: t-SNE, UMAP

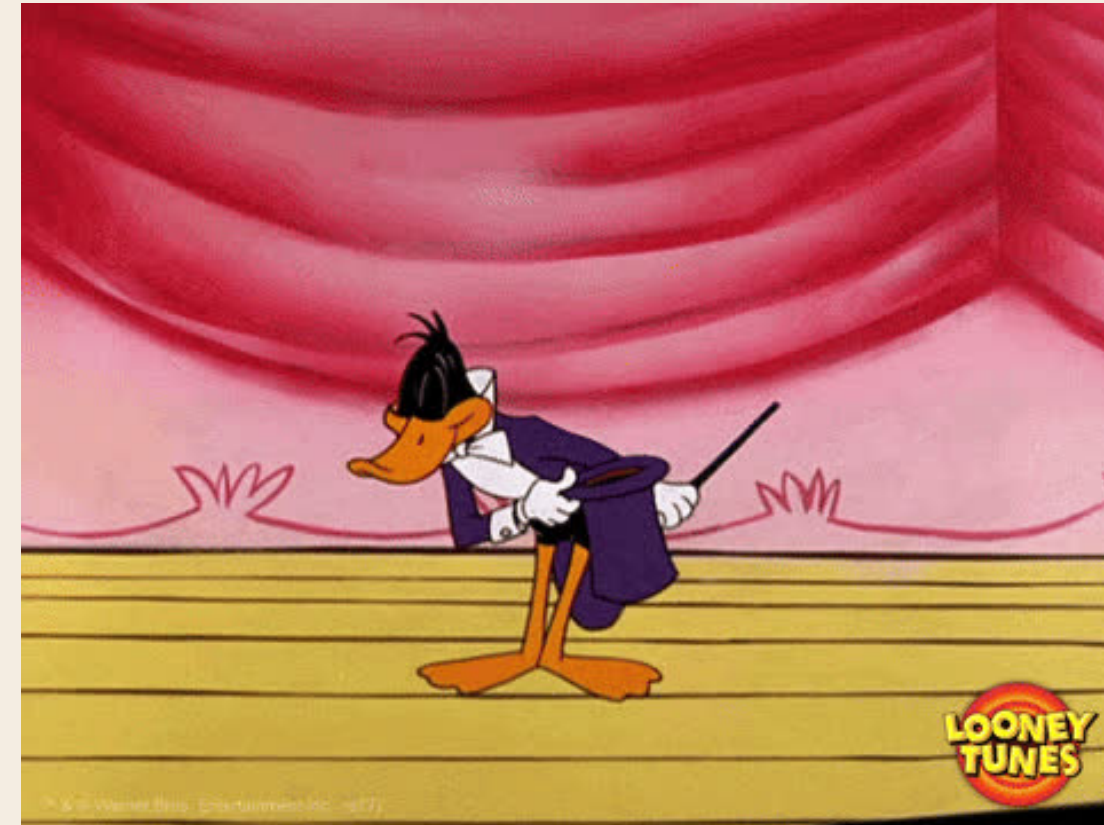


K-means vs GMM

Two representative techniques are k-means and Gaussian Mixture Model (GMM). K-means assigns data points to the nearest clusters, while GMM assigns data to the Gaussian densities that best represent the data.



THANK YOU!



Weekly Slides & Code will be shared on Github (star it for
bookmark):

<https://github.com/kr7001/DSA1101-teaching-materials>